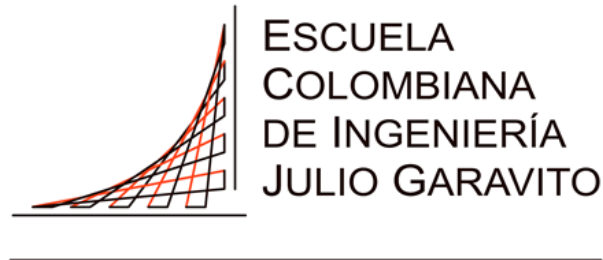


**UNIVERSIDAD ESCUELA COLOMBIANA DE  
INGENIERÍA JULIO GARAVITO**



---

**UNIVERSIDAD**

**INFORME DE ARQUITECTURA**  
**ECI-Commerce**  
**Equipo Arquitectos**

**AUTORES**

Andrés Felipe Chavarro Plazas  
Carlos David Barrero Velásquez  
Emily Noreña Cardozo  
Jesús Alberto Jauregui Conde  
Roger Alenxander Rodríguez Abril  
Sebastián Julián Villarraga Guerrero

**PROGRAMA**

Ingeniería de Sistemas

**ASIGNATURA**

Ciclos de Vida del Desarrollo de Software (CVDS)

**PROFESORES**

Ing. Rodrigo Humberto Gualtero Martínez  
Ing. Andrés Martín Cantor Urrego

**BOGOTÁ DC, COLOMBIA**  
Miércoles, 16 de Abril de 2025

## Generalidades

---

En el desarrollo de sistemas modernos, la elección y aplicación de tecnologías, patrones de arquitectura, bases de datos y herramientas debe responder a una visión estratégica, donde cada decisión esté alineada con los objetivos del negocio, la escalabilidad y la productividad del equipo. No se trata de adoptar tecnologías por tendencia, sino de utilizar aquellas que mejor se adapten a las necesidades específicas de cada módulo, considerando factores como los tiempos de entrega, el conocimiento del equipo y la interoperabilidad entre componentes. Cada elección debe aportar valor, mantener la calidad del software y permitir un desarrollo sostenible en el tiempo. Por lo que se plantearon las siguientes generalidades para cada uno de los módulos presentes en este proyecto.

### Uso de Springboot

En un entorno de desarrollo con múltiples equipos trabajando sobre una arquitectura distribuida como microservicios, es clave tomar decisiones que aseguren productividad, calidad del código, y colaboración fluida. Por eso, elegimos Spring Boot como nuestro stack backend común, por su enfoque en este tipo de arquitectura y los siguientes motivos:

- **Dominio del Lenguaje y Framework:** Todo el equipo ya conoce Java y Spring Boot. Esto significa que podemos enfocarnos directamente en desarrollar la lógica de negocio, sin perder tiempo aprendiendo nuevas herramientas. Además, Se mantiene la calidad del código, ya que todos dominamos las buenas prácticas, patrones de diseño e información del proyecto para asegurar un código limpio.
- **Optimización del Tiempo de Desarrollo:** Al no tener que aprender otro lenguaje o stack desde cero, podemos cumplir con los tiempos de entrega de los microservicios, pues no corremos el riesgo de errores por desconocimiento de tecnologías nuevas y aprovechamos el ecosistema Spring, que ya tenemos integrado: seguridad, pruebas, control de errores, manejo de dependencias, etc.
- **Soporte entre Squads:** Dado que todos usamos el mismo stack, cualquier developer puede entrar a ayudar a otro equipo en caso de emergencia o sobrecarga de tareas. De este modo, se puede rotar personal entre squads sin que esto implique una caída en la velocidad o

calidad, y se crea una cultura común de desarrollo: mismos patrones, mismas herramientas, mismas prácticas.

## Bases de Datos

En el desarrollo de aplicaciones modernas, especialmente bajo arquitecturas de microservicios, la elección entre bases de datos SQL y NoSQL no tiene por qué ser excluyente. Cada tipo de base de datos ofrece ventajas específicas según el tipo de módulo o funcionalidad que se esté construyendo. A continuación, se presenta un cuadro comparativo que ilustra las diferencias clave entre ambas tecnologías.

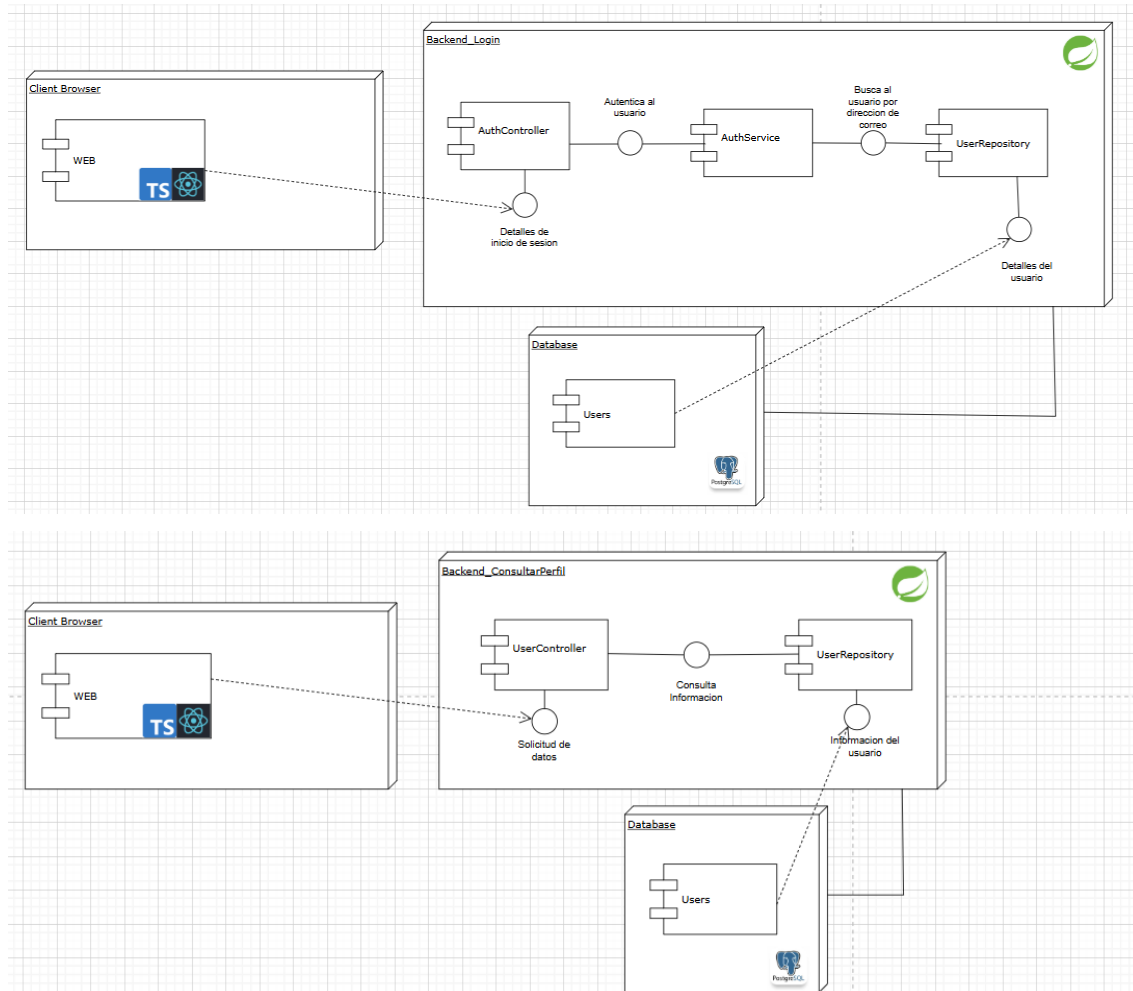
| Característica / Criterio       | SQL (Relacional)                                                                                                                                                                                                                               | NoSQL (No Relacional)                                                                                                              |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Estructura de datos</b>      | Tablas con relaciones definidas, esquemas fijos                                                                                                                                                                                                | Documentos, grafos, pares clave-valor o columnas; esquemas flexibles                                                               |
| <b>Transacciones ACID</b>       | Soporte total, es ideal para lógica financiera pues hay un alto nivel de precisión y confiabilidad, usando principios de atomicidad de transacciones, consistencia en los datos, aislamiento en las transacciones y registro de transacciones. | Soporte limitado o eventual, pues depende de la capacidad del motor utilizado, como MongoDB o Cassandra.                           |
| <b>Escalabilidad</b>            | Vertical (más recursos al servidor)                                                                                                                                                                                                            | Horizontal (añadir más nodos fácilmente)                                                                                           |
| <b>Flexibilidad del esquema</b> | Bajo, ya que se usan estructuras de datos fijas que dificultan la mutación de las tablas de datos.                                                                                                                                             | Alto, ya que no hay una estructura determinada y cada documento, grafo o esquema varia su forma de organizar los datos insertados. |

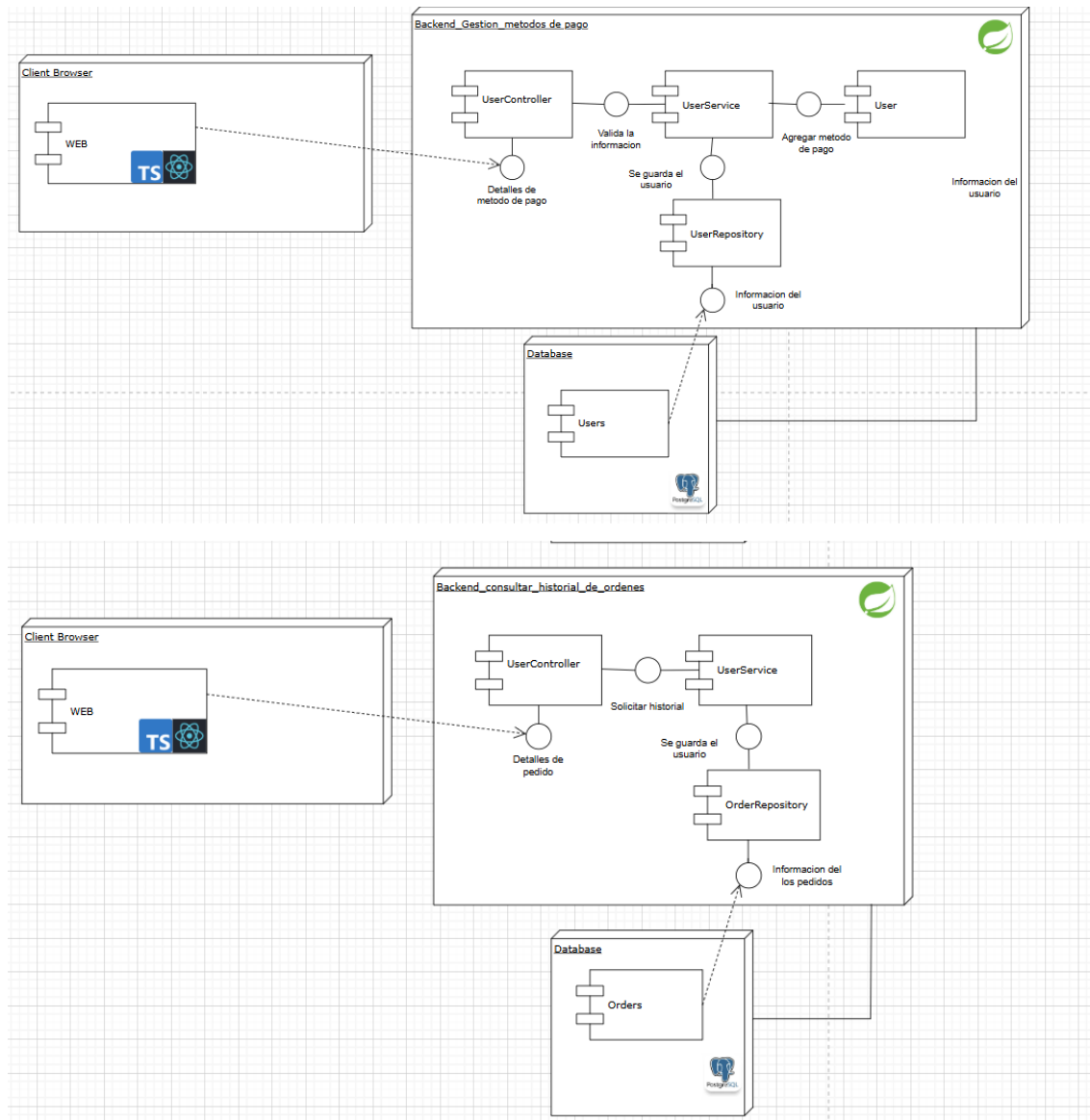
|                                      |                                                                                                                                                                             |                                                                                                                                                                                                                                           |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Velocidad en escritura masiva</b> | Moderada, requiere validaciones para alterar las tablas o realizar transacciones.                                                                                           | Alta, se puede almacenar o variar grandes cantidades de datos en poco tiempo                                                                                                                                                              |
| <b>Consulta de datos complejos</b>   | Potente, pues cuenta con SQL estándar, JOINS, agregaciones, vistas y demás                                                                                                  | Limitada o especializada, pero escalable. No cuentan con herramientas complejas a comparación de las SQL                                                                                                                                  |
| <b>Casos de uso ideales</b>          | <ul style="list-style-type: none"> <li>- Pagos y transacciones</li> <li>- Facturación</li> <li>- Gestión de usuarios (con roles y permisos)</li> <li>- Auditoría</li> </ul> | <ul style="list-style-type: none"> <li>- Historial de actividad del usuario</li> <li>- Feed o timeline social</li> <li>- Datos de sensores / IoT</li> <li>- Contenido dinámico y cambiante (ej. posts, comentarios, productos)</li> </ul> |

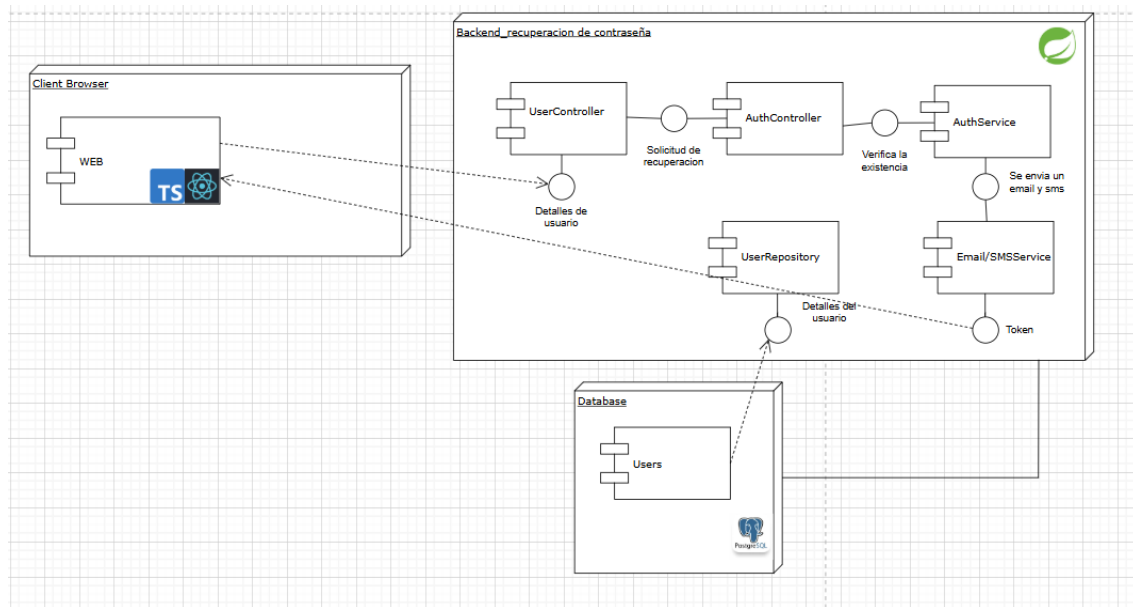
Es común poder usar ambas en un mismo servicio, pues nos permite escoger lo mejor de ambos tipos de bases y sacar mayor rendimiento en el manejo de los datos. Esta información brindada, sirve como punto de partida para cada que squad decida como manejar los datos dentro de su sistema.

## Módulo Gestión de Usuarios

La arquitectura del backend fue diseñada con un enfoque pragmático y orientado a la mantenibilidad, donde la organización modular, la calidad del código y la integridad de los datos son prioridades. Por este motivo, se utilizó **SpringBoot** como framework principal, facilitando una integración limpia con el ecosistema de datos y permitiendo un desarrollo ágil y robusto. Además, realizamos el flujo de componentes de la siguiente forma basados en cada una de las solicitudes (iniciar sesión consultar perfil, Autenticación, Recuperar contraseña, gestionar método de pago, consultar historial de ordenes):



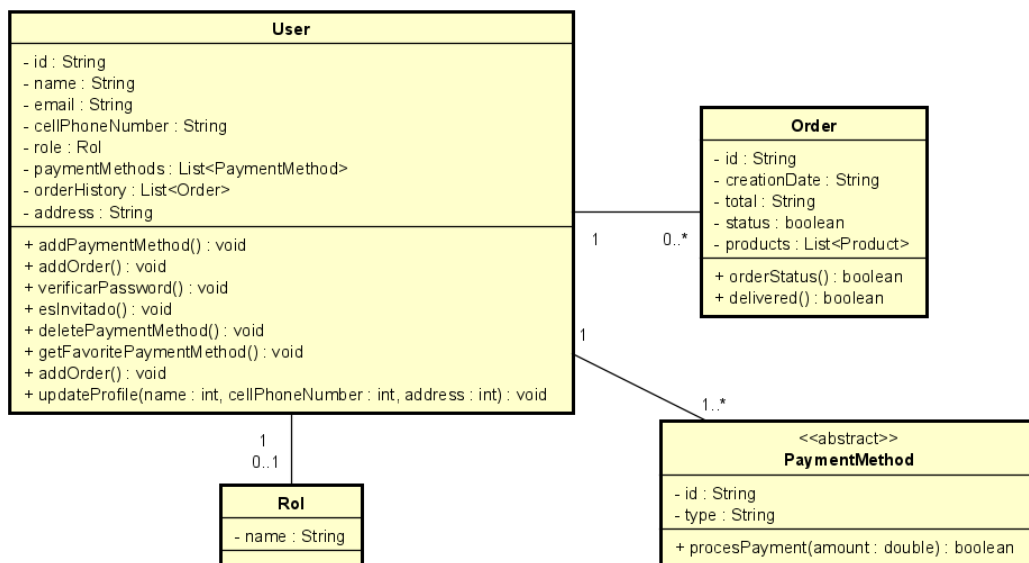




Dado que este módulo está enfocado en la gestión de usuarios, incluyendo el registro, autenticación segura con JWT, asignación de roles únicos (estudiante, profesor, administrativo, invitado y distintos tipos de administradores), manejo de perfil, métodos de pago preferidos y recuperación de contraseña, optamos por utilizar una base de datos SQL.

Esta elección se fundamenta en su robusto soporte a relaciones entre entidades como Usuario, Rol, Orden y Método de Pago, así como su capacidad para manejar consultas estructuradas y validaciones de integridad referencial. Además, el cumplimiento de los principios ACID permite garantizar la consistencia y seguridad en operaciones críticas como el inicio de sesión, la actualización de información personal o la recuperación de contraseñas.

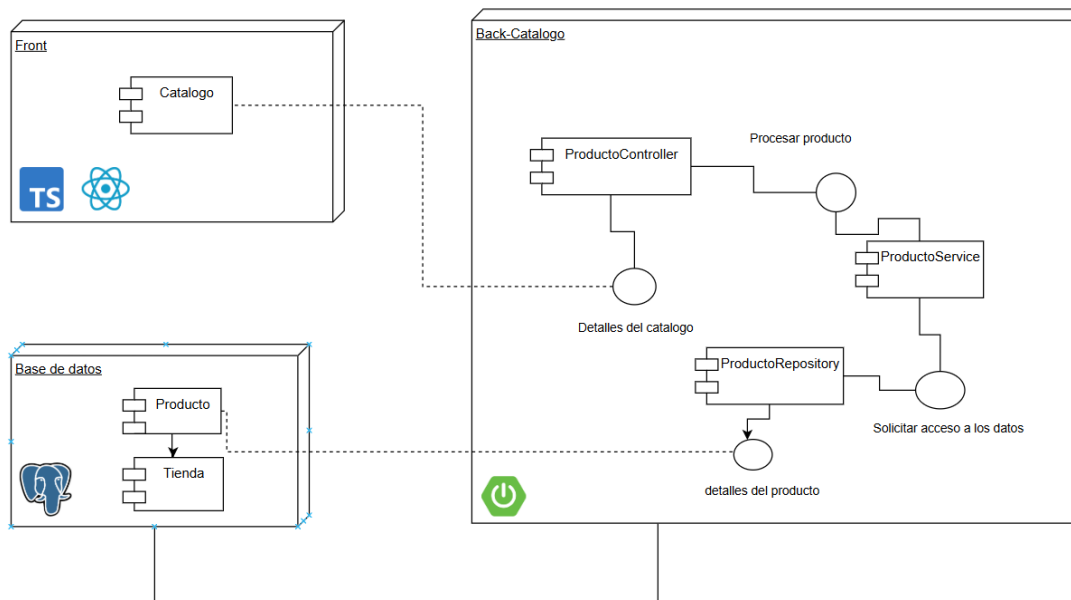
La combinación de SpringBoot con una base de datos SQL nos permite desarrollar un módulo fiable, seguro y altamente mantenible, asegurando una correcta autenticación y autorización del usuario, así como una experiencia fluida en la gestión de su información personal. A continuación, se presenta el modelo de clases propuesto para comprender su estructura interna.





## Módulo Catálogo de Productos

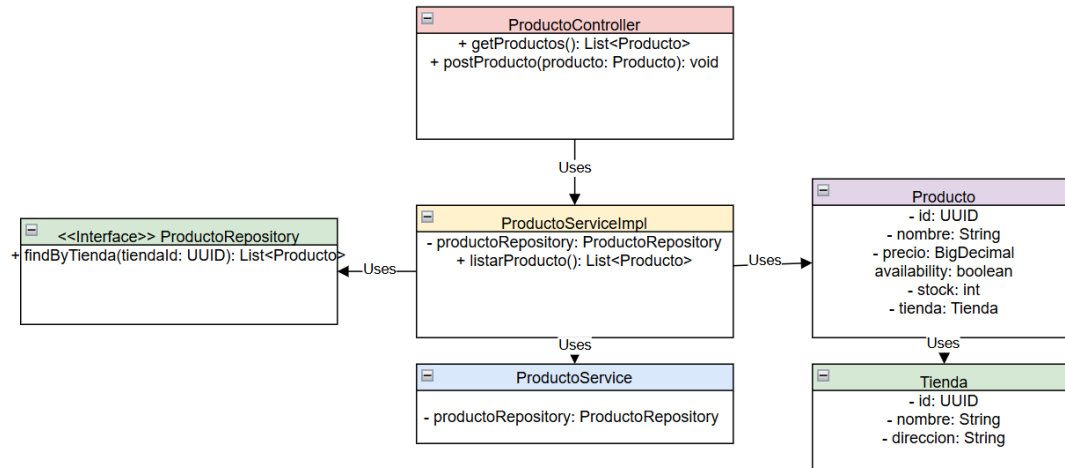
La arquitectura del backend fue diseñada con un enfoque pragmático y orientado a la mantenibilidad, donde la organización modular, la calidad del código y la integridad de los datos son prioridades. Por este motivo, se utilizó **Spring Boot** como framework principal, facilitando una integración limpia con el ecosistema de datos y permitiendo un desarrollo ágil y robusto. Además, realizamos el flujo de componentes de la siguiente forma:



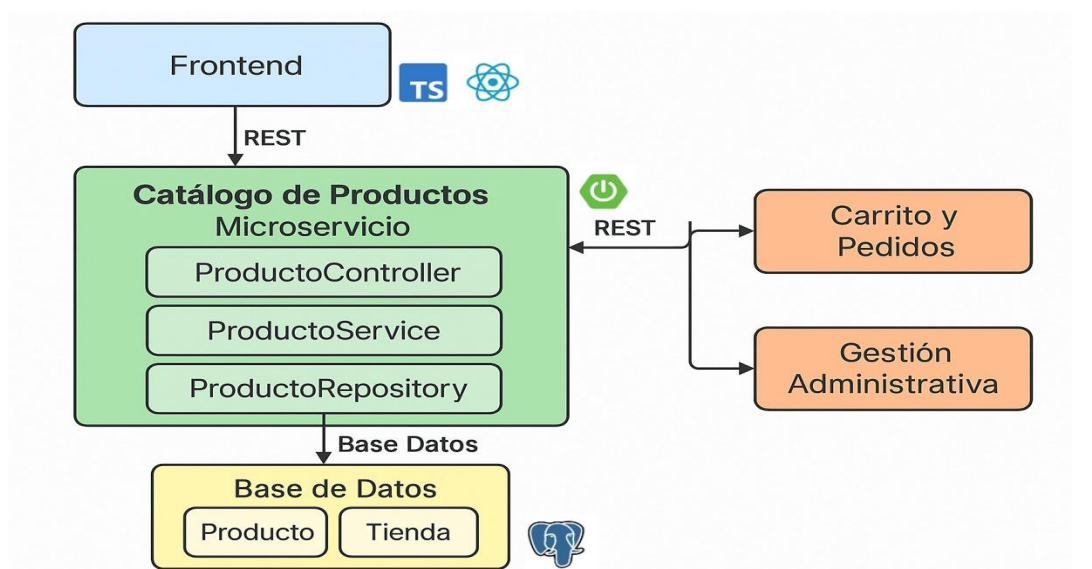
Dado que este módulo está enfocado en la gestión y visualización de un catálogo de productos, con separación por categorías (papelería, libros, cafetería y kioscos), y funcionalidades como búsqueda avanzada, filtros dinámicos, y stock en tiempo real, por esa razón optamos por utilizar como base de datos una **SQL**.

Esta fue elegida por su fuerte soporte a relaciones entre entidades, su capacidad para manejar consultas complejas de manera eficiente y su cumplimiento con los principios ACID, lo cual garantiza la confiabilidad en operaciones críticas como la actualización de stock o la validación de disponibilidad de productos.

Esta combinación entre **Spring Boot** y **SQL** nos permite construir un módulo sólido, altamente mantenible y con excelente rendimiento, asegurando que los productos estén siempre accesibles, actualizados y disponibles según la demanda de los usuarios. Por otro lado, planteamos el siguiente modelo de clases para comprender su estructuración interna.



El módulo de Catálogo de Productos se relaciona directamente con el módulo de Carrito y Pedidos, ya que este último necesita consultar la información del catálogo para mostrar detalles actualizados de los productos, verificar el stock en tiempo real y permitir que los usuarios agreguen, modifiquen o confirmen sus pedidos correctamente. También mantiene una relación directa con el módulo de Gestión Administrativa, que se encarga de realizar operaciones de alta, baja y modificación de productos, así como de gestionar el stock disponible. Estas interacciones aseguran que la información del catálogo esté siempre actualizada y sincronizada con las operaciones de compra y administración. Esto se puede ver de tal forma.



## Módulo Carrito de Compras y Pedidos

---

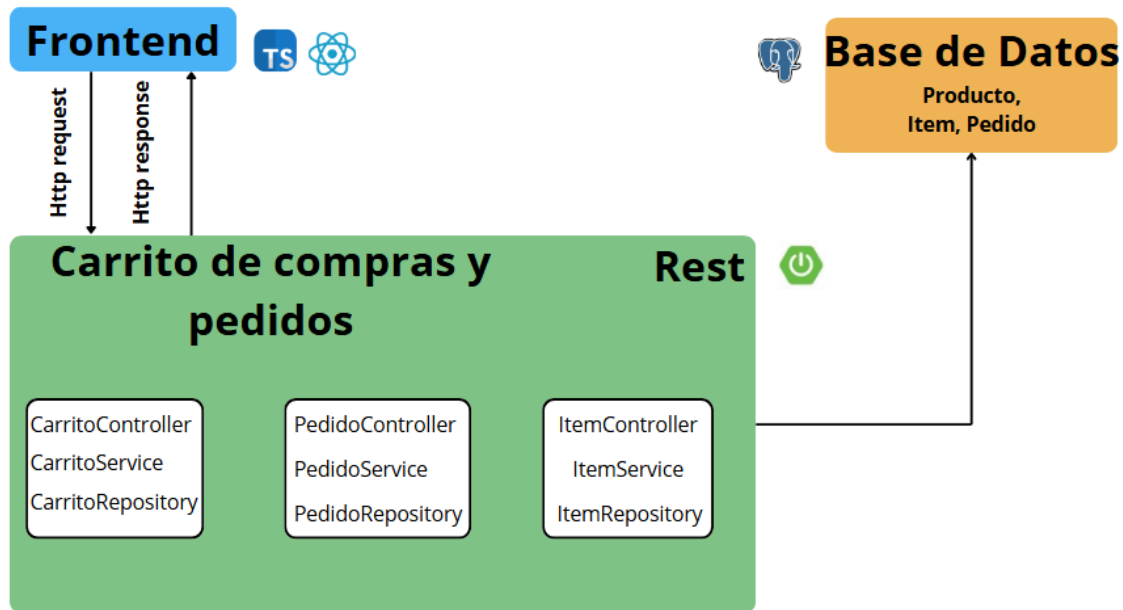
Nuestra solución para el módulo de Carrito de Compras y Pedidos fue diseñada con un enfoque en escalabilidad controlada y consistencia de datos, utilizando Spring Boot como framework principal para aprovechar su ecosistema integrado y capacidades de producción listas. El diseño prioriza:

1. Integridad transaccional en operaciones críticas (gestión de stock, confirmación de pedidos)
2. Desacoplamiento modular entre componentes clave
3. Trazabilidad completa del ciclo de vida del pedido

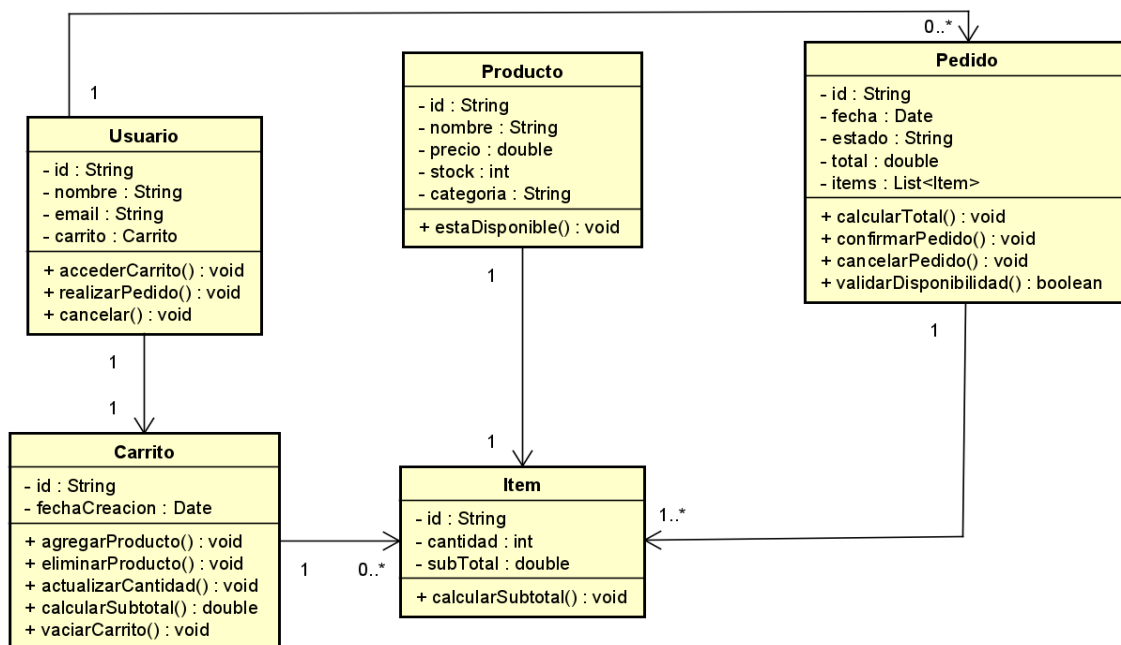
Base de Datos: PostgreSQL

1. Garantía ACID para:
  - Operaciones críticas de gestión de stock concurrente (evita sobreventa)
  - Transacciones distribuidas al confirmar pedidos (actualizar carrito + crear pedido + modificar inventario)
2. Ventajas Específicas para el dominio:
  - CHECK constraints para validar:  $\text{stock} \geq 0$ , cantidades  $> 0$  (Reglas RN2, RN3)
  - Foreign Keys que garantizan: productos existentes en pedidos, usuario válido en carrito
  - Transacciones anidadas para procesos como "Agregar al carrito  $\rightarrow$  Verificar stock  $\rightarrow$  Confirmar"

Presentamos el flujo de componentes:

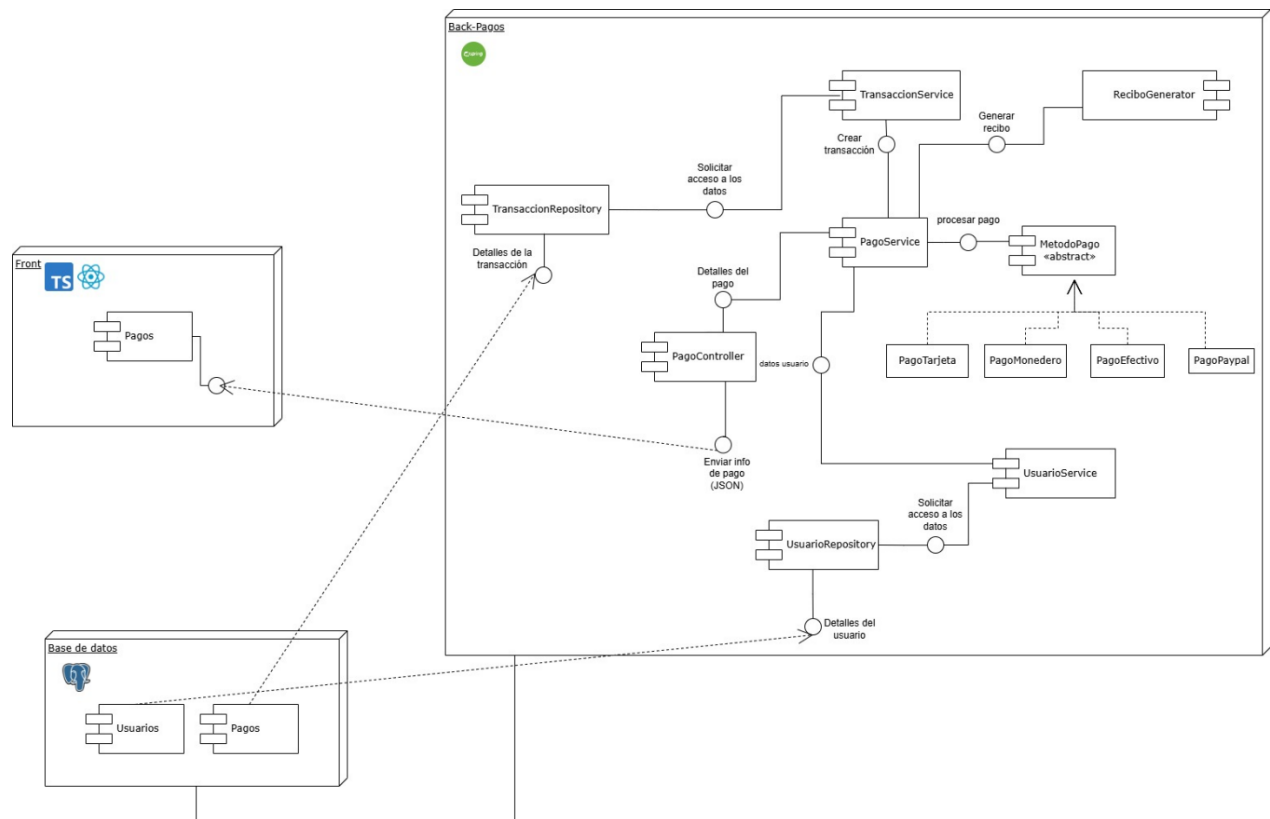


En este modelo de arquitectura se omite el componente producto ya que es modelado en el módulo anterior. Por otro lado, planteamos el siguiente modelo de clases para comprender su estructuración:

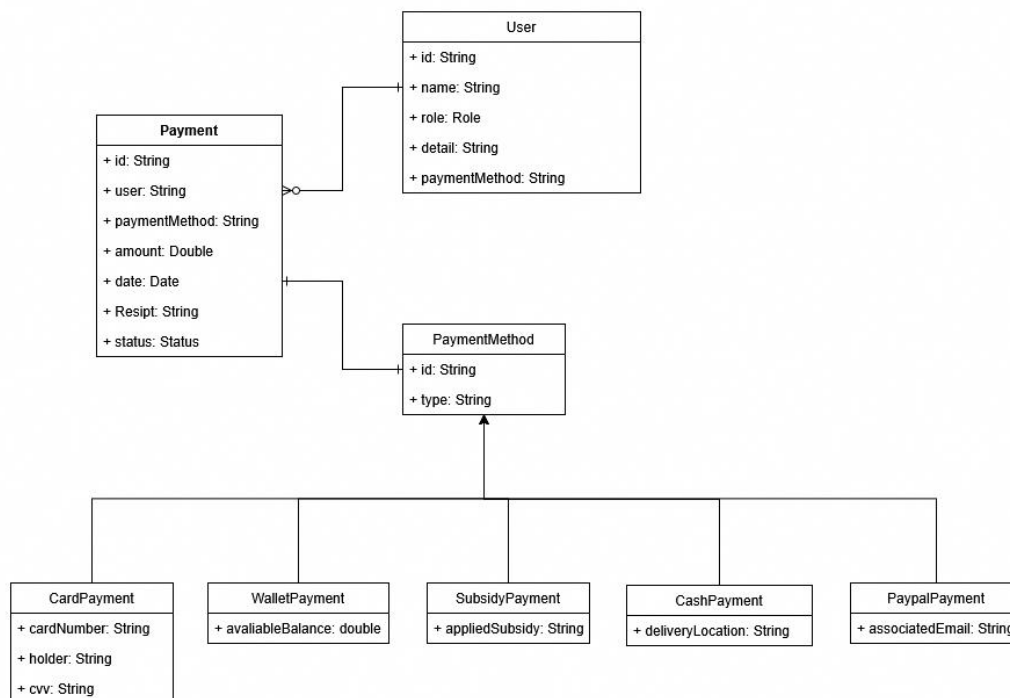


## Módulo de Pagos y Monedero Digital

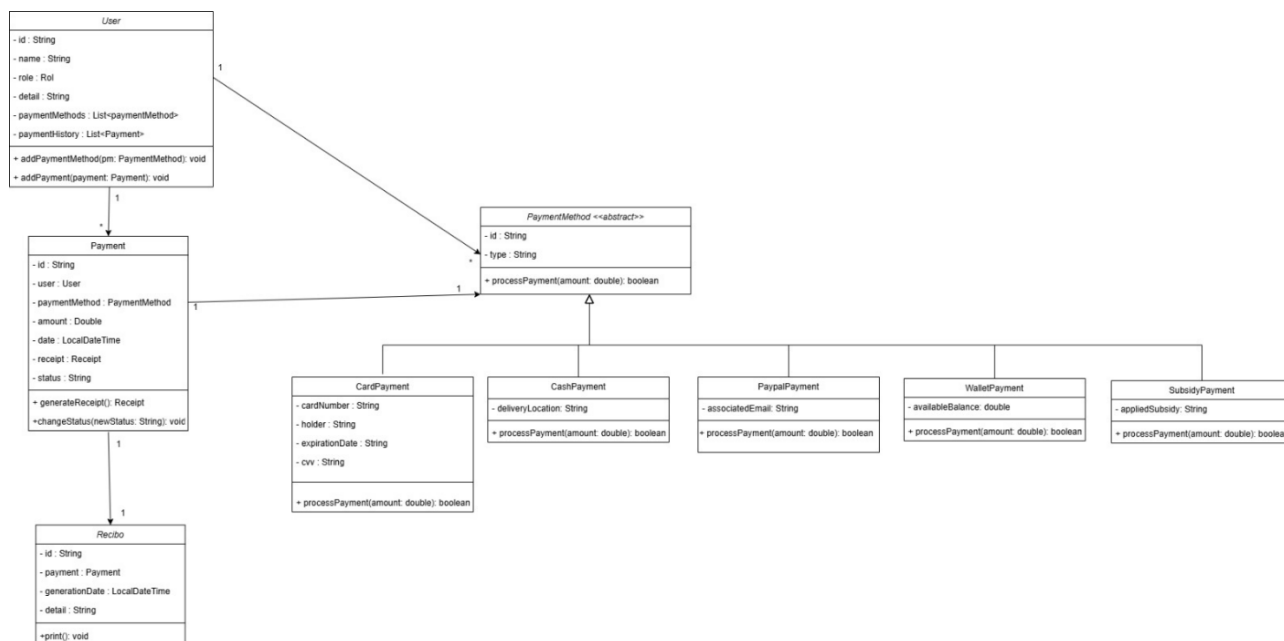
La arquitectura de este backend fue diseñada con un enfoque pragmático y alineado a las necesidades del sistema, donde la confiabilidad y la calidad del código. Por este motivo, usamos Springboot como framework teniendo en cuenta lo ya mencionado en este archivo. Además, realizamos el flujo de componentes de la siguiente forma:



Dado que este módulo está enfocado en la gestión de transacciones financieras y un monedero virtual, tomamos la decisión de utilizar una base de datos SQL, ya que necesitamos garantizar consistencia, integridad y soporte completo para transacciones ACID. Por lo cual, Postgresql fue la opción que mejor se nos adaptaba para cumplir con las especificaciones del módulo, ya que este cuenta con funciones de tipos y transacciones que nos asegura que las ACID se concreten de la forma ideal.



Esta elección nos permite operar con confianza en escenarios críticos, como pagos, recargas y movimientos monetarios, donde cada operación debe ser precisa, trazable y segura. Por otro lado, planteamos el siguiente modelo de clases para comprender su estructuración interna y poder realizar los modelos del diagrama anterior, aunque es bastante parecido.



## Módulo de Notificaciones

---

Este módulo estará desacoplado del core del sistema mediante una arquitectura basada en eventos. Cada vez que ocurra una acción relevante (pedido creado, estado actualizado, reserva confirmada), el sistema publicará un evento en un bus de mensajes, y el Servicio de Notificaciones lo consumirá.

Comunicación:

Protocolo: HTTP y RabbitMQ/Kafka.

Estilo: RESTful APIs + Mensajería Asíncrona (event-driven)

Tecnologías:

- RabbitMQ o Apache Kafka: para recibir eventos del sistema
- JavaMailSender o SendGrid API: para enviar correos}
- Firebase Cloud Messaging (FCM): para notificaciones push

Base de Datos: MongoDB (NoSQL)

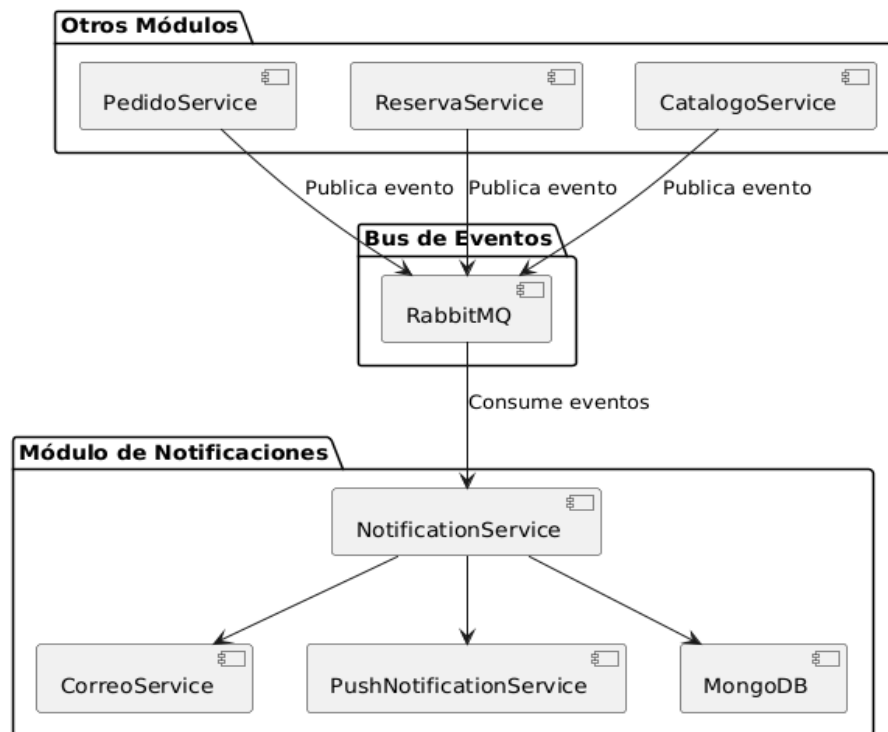
Justificación:

- Alta velocidad de escritura para logs de notificaciones
- No se requiere consistencia estricta
- Flexibilidad en el almacenamiento de distintos tipos de notificaciones

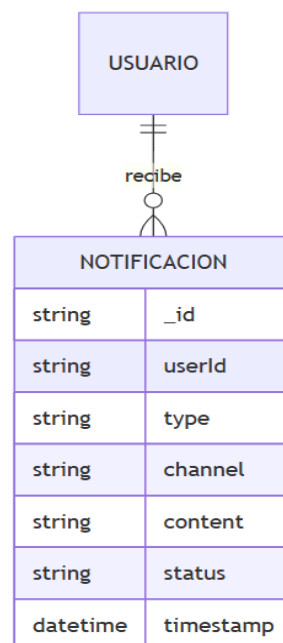
[MODELO DE COLECCIÓN]

```
{ "_id": "uuid",  
  "userId": "user-id",  
  "type": "EMAIL | PUSH",  
  "channel": "pedido_confirmado | estado_actualizado | recordatorio",  
  "content": "Texto del mensaje enviado",  
  "status": "ENVIADO | FALLIDO",  
  "timestamp": "2025-04-09T12:00:00Z"  
}
```

[MODELO DE COMPONENTES]

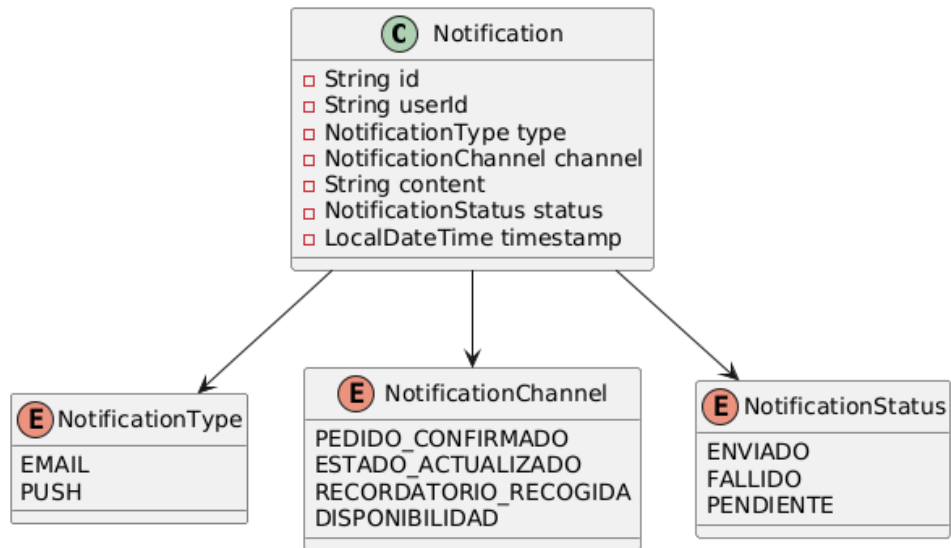


[MODELO DE BASE DE DATOS]

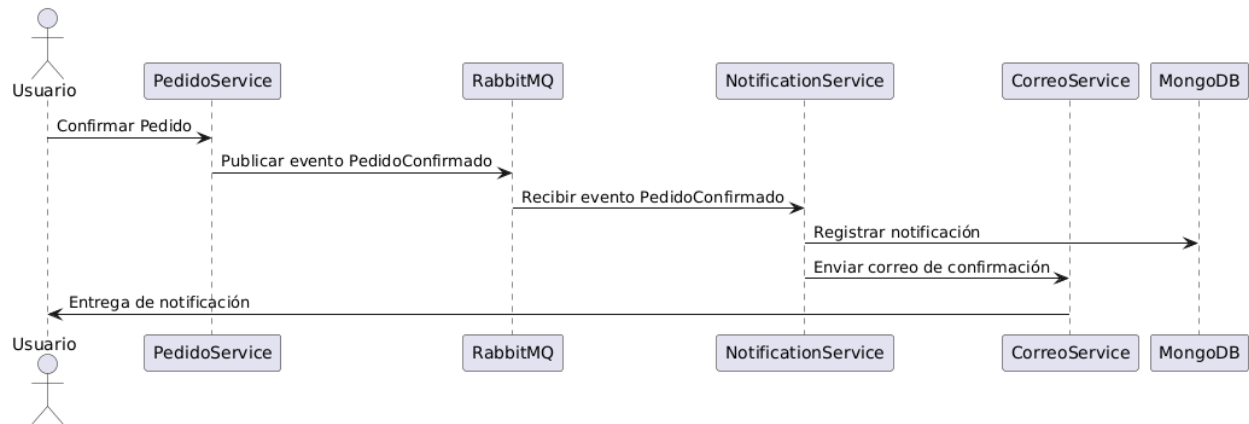




[DIAGRAMA DE CLASES]

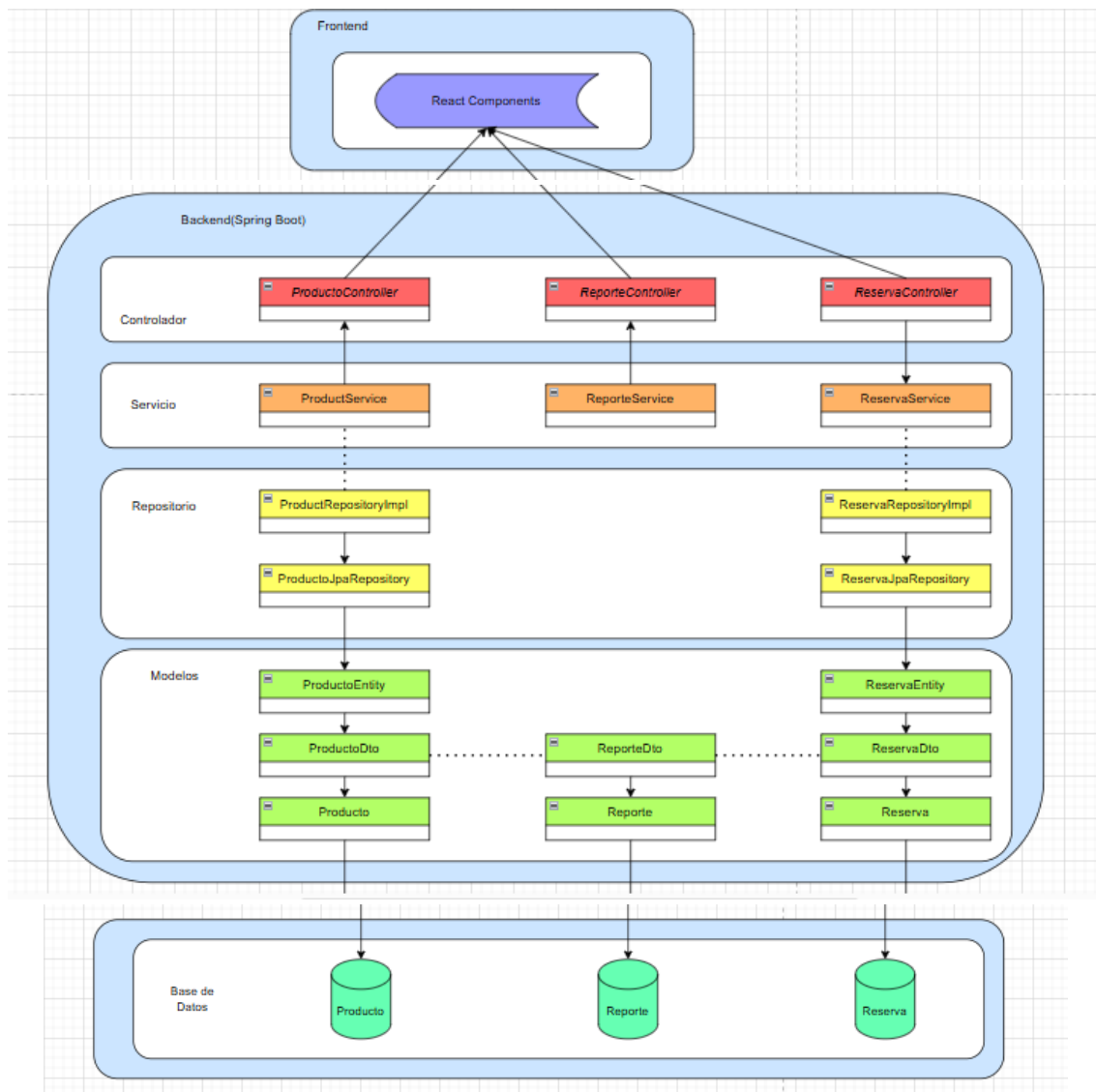


[Diagrama de Secuencia] (Notificación por evento de pedido confirmado)



## **Módulo de Gestion Administrativa**

La arquitectura del módulo de Gestión Administrativa fue diseñada utilizando un enfoque pragmático y alineado con los requisitos del sistema, priorizando la confiabilidad, la calidad del código y la escalabilidad del componente. En este contexto, se optó por el uso de Spring Boot como framework principal, facilitando la configuración, la modularización y la integración de servicios dentro del ecosistema del proyecto.



Dado que el módulo de Gestión Administrativa está orientado al manejo de productos, control de reservas y generación de reportes, se requiere una gestión confiable y estructurada de la información. Por ello, se optó por una base de datos relacional SQL, ya que proporciona las garantías necesarias de consistencia, integridad y soporte completo para transacciones ACID.

Entre las posibles opciones, PostgreSQL fue seleccionada por su robustez, su soporte para tipos de datos personalizados y su capacidad de manejar operaciones transaccionales complejas. Estas características aseguran que las operaciones administrativas –como la creación, actualización o cancelación de reservas, así como el registro de productos y generación de reportes– se ejecuten de manera segura, coherente y con alta confiabilidad.